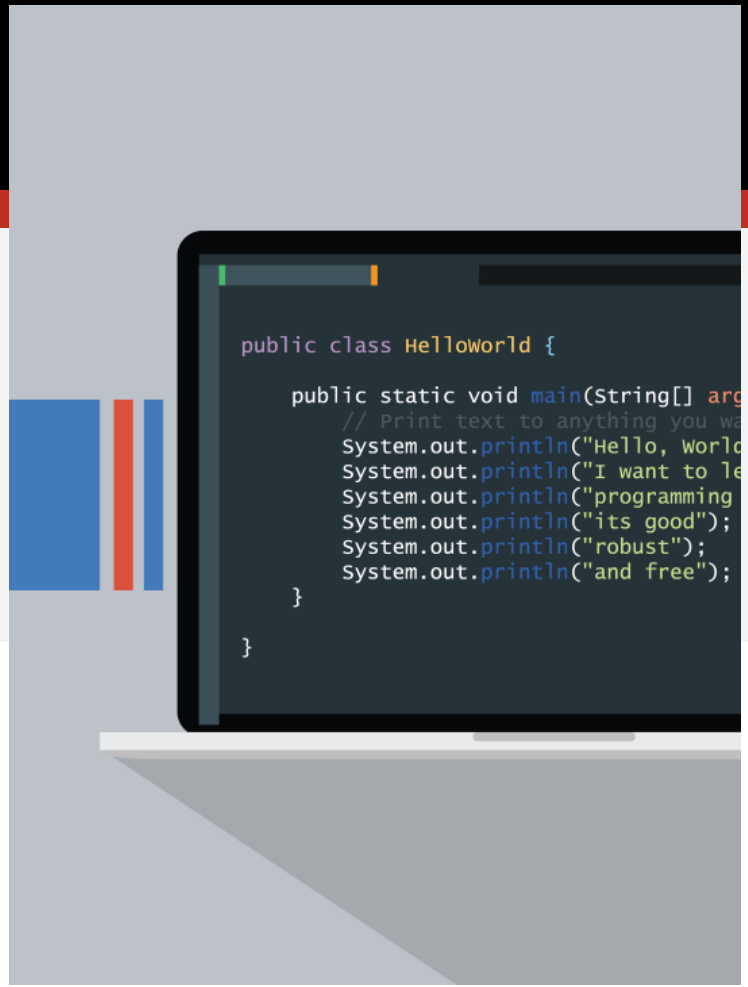
MONTCLAIR
STATE UNIVERSITY

CSITBG2 – Data Structures and Algorithms: Java



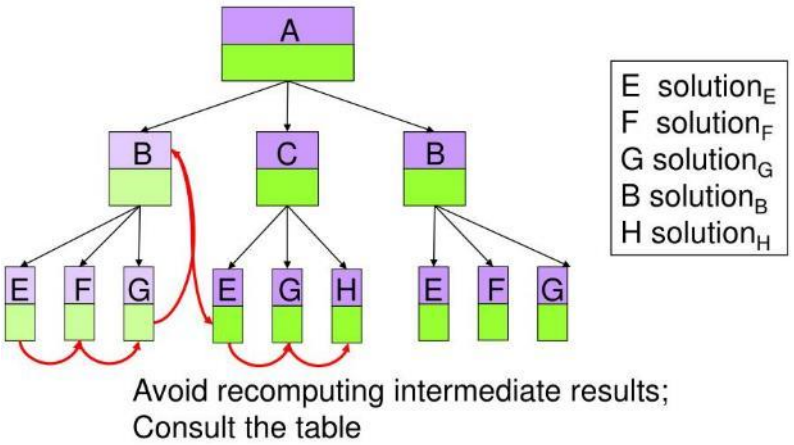
Dynamic Programming

Table of Contents

- **Dynamic Programming**
- Rod-cutting Problem
- Longest Common Subsequence Problem

Dynamic Programming

- Dynamic Programming (DP) is a powerful algorithmic technique used to solve optimization problems by **breaking them down** into simpler subproblems and **storing the results** of those subproblems to avoid redundant computation.
- It's especially effective for problems with:
  - **overlapping subproblems**
  - **optimal substructure**



Problem-solving Paradigms

Divide-and-conquer paradigm:

- Break the problem into several **disjoint** subproblems that are similar to the original one but smaller in size.
- Solve the subproblems recursively.
- Combine these solutions to create a solution to the original problem.

Dynamic programming:

- Break the problem into several **overlapping** subproblems that are similar to the original one but smaller in size.
- Solve the subproblems recursively **only once**.
- Combine these solutions to create a solution to the original problem.

Example Comparison

- **Fibonacci Sequence**

$$F(n) = \begin{cases} 0 & if\ n = 0 \\ 1 & if\ n = 1 \\ F(n-1) + F(n-2) & if\ n > 1 \end{cases}$$

- **Divide and Conquer (Recursion)**
  - Time: Exponential ($O(n^2)$)

```
int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);
// Computes same values
repeatedly
}
```

- **Dynamic Programming**
  - Time: Linear ($O(n)$)

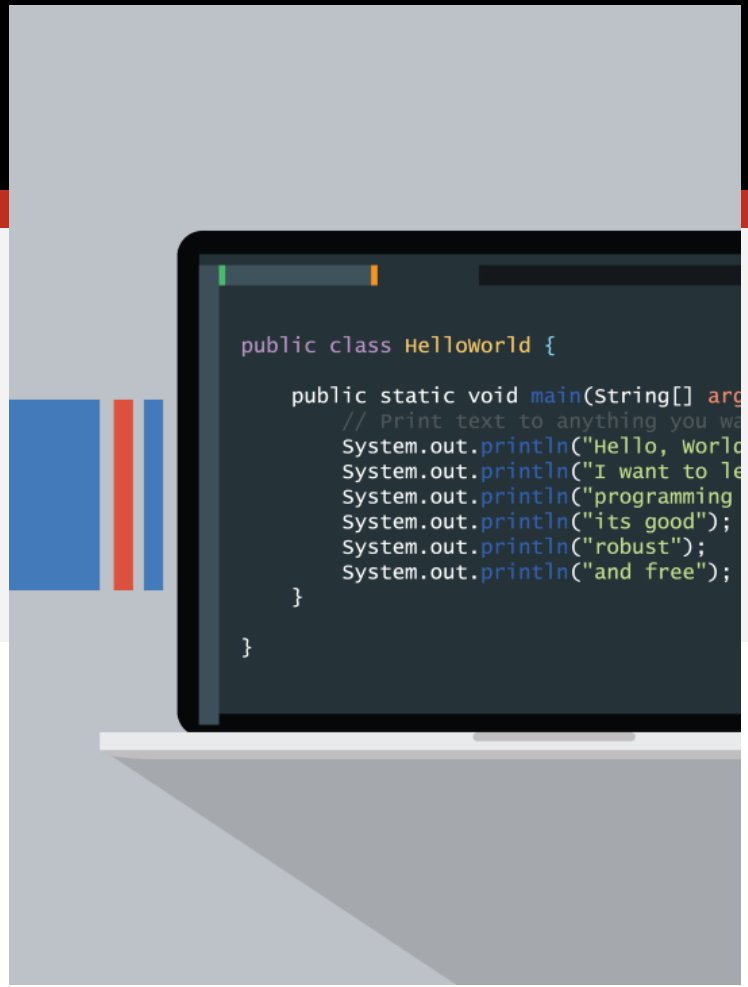
```
int fib(int n) {
    int[] dp = new int[n+1];
    dp[0] = 0; dp[1] = 1;
    for (int i = 2; i <= n; i++)
        dp[i] = dp[i-1] + dp[i-2];
// Uses stored results
    return dp[n];
}
```

DP Algorithm Design

- When develop a dynamic-programming algorithm, we follow a sequence of four steps:
  - Characterize the structure of an optimal solution
  - Recursively define the value of an optimal solution
  - Compute the value of an optimal solution, typically in a bottom-up fashion
  - Construct an optimal solution from computed information

MONTCLAIR
STATE UNIVERSITY

CSITBG2 – Data Structures and Algorithms: Java



Rod-cutting Problem

Table of Contents

- Dynamic Programming
- **Rod-cutting Problem**
- Longest Common Subsequence Problem

Rod-cutting Problem

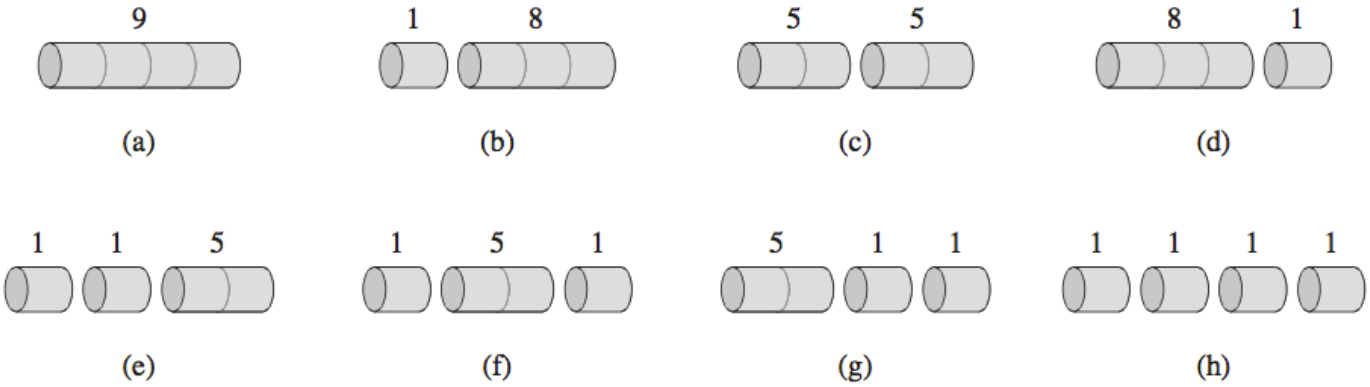
- The Rod-Cutting Problem is a classic example used to illustrate dynamic programming techniques like optimal substructure and overlapping subproblems.

Problem Statement

- Given a rod of length n inches and a table of prices p_i for $i = 1,2, \ldots, n$, determine the maximum revenue r_n obtainable by cutting up the rod and selling the prices.

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30

The 8 possible ways of cutting up a rod of length 4



Optimal Substructure

- More generally, we can frame the values r_n for $n \geq 1$ in terms of optimal revenues from shorter rods:

$$r_n = \max(p_n, r_1 + r_{n-1}, r_2 + r_{n-2}, \dots, r_{n-1} + r_1)$$

$where$ p_n: the revenue from no cut at all

$r_i + r_{n-i}$: the revenue from cutting at the i-th place

- This is called the **optimal substructure property.**
- Optimal solutions to a problem incorporate optimal solutions to related subproblems, which we may solve independently.

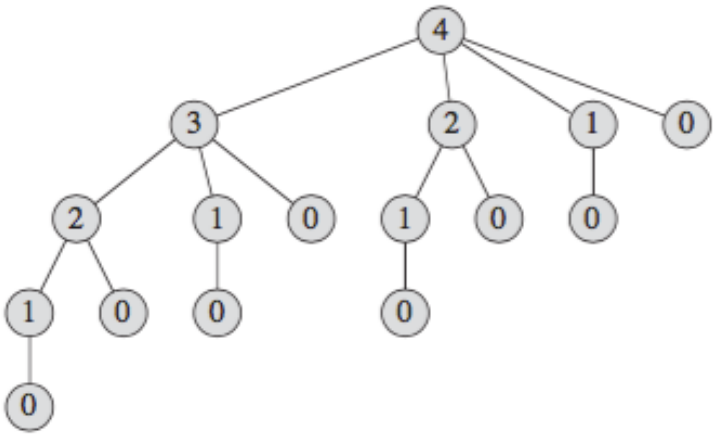
Recursively Definition

- To make is simpler, we have

$$r_n = \max_{1 \le i \le n} (p_i + r_{n-1})$$

  - Only the right-hand side of a cut at the i-th place will be cut further.
- Based on the previous formula, we derive the following recursive algorithm (Time complexity: $O(2^n)$):

```
CUT-ROD(p, n)
if n == 0
    return 0
q = -∞
for i = 1 to n
    q = max(q, p[i] + CUT-ROD(p, n - i))
return q
```



The recursion tree showing recursive calls resulting from a call CUT-ROD(p,n) for $n = 4$.

Calculating Solutions: Top-down Approach

- In dynamic programming, we only need to **calculate them once**.
  - The first time we calculate r_i, we store it in a place.
  - The next time we need to use r_i, we just retrieve it directly.
- `MEMOIZED-CUT-ROD-AUX(p, n, r)` calculates r_n in a top-down fashion by recursion.

```
MEMOIZED-CUT-ROD(p, n)
let r[0..n] be a new array
for i = 0 to n
    r[i] = -∞
return MEMOIZED-CUT-ROD-AUX(p, n, r)
```

```
MEMOIZED-CUT-ROD-AUX(p, n, r)
if r[n] ≥ 0
    return r[n]
if n == 0
    q = 0
else q = -∞
    for i = 1 to n
        q = max(q, p[i] + MEMOIZED-CUT-ROD-AUX(p, n - i, r))
r[n] = q
return q
```

Calculating Solutions: Bottom-up Approach

- Most often, we want a bottom-up solution for two reasons:
  - Logically, it is easy to follow.
  - Faster, as it avoids recursion.

```
BOTTOM-UP-CUT-ROD(p, n)
let r[0..n] be a new array
r[0] = 0
for j = 1 to n
    q = -∞
    for i = 1 to j
        q = max(q, p[i] + r[j - i])
    r[j] = q
return r[n]
```

Time complexity: $O(n^2)$
Space complexity: $O(n)$
Dynamic programming is an approach to trading space for efficiency.

Solution Reconstruction

```
EXTENDED-BOTTOM-UP-CUT-ROD(p, n)
let r[0..n] and s[0..n] be new arrays
r[0] = 0
for j = 1 to n
    q = -∞
    for i = 1 to j
        if q < p[i] + r[j - i]
            q = p[i] + r[j - i]
            s[j] = i
    r[j] = q
return r and s
```

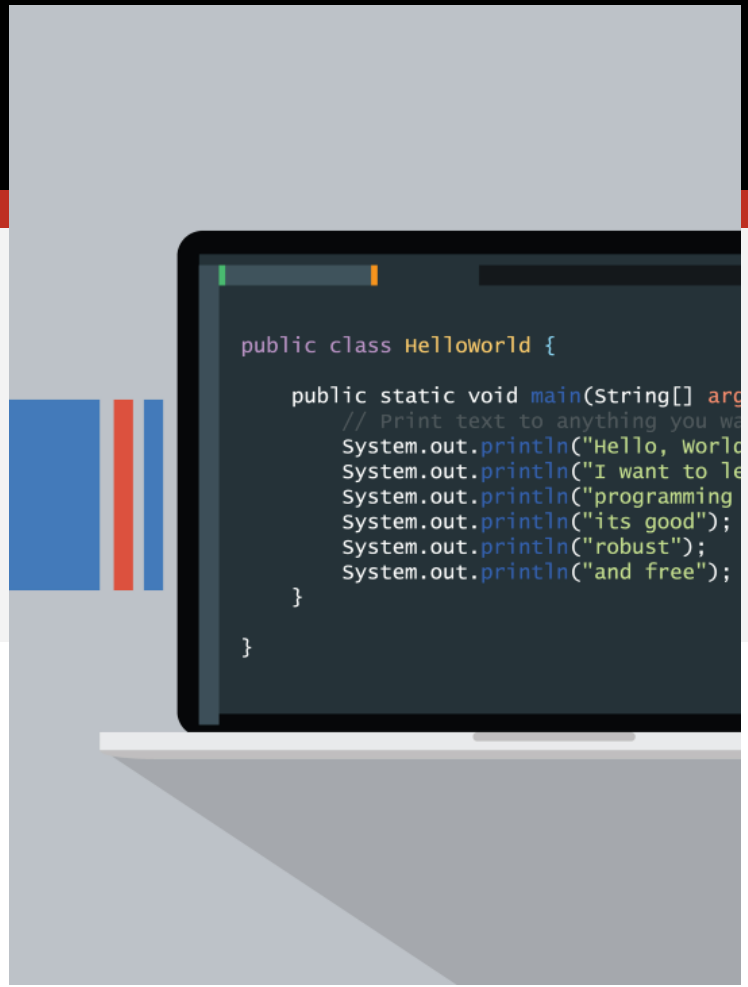
```
PRINT-CUT-ROD-SOLUTION(p, n)
(r, s) = EXTENDED-BOTTOM-UP-CUT-ROD(p, n)
while n > 0
    print s[n]
    n = n - s[n]
```

In our rod-cutting example, the call EXTENDED-BOTTOM-UP-CUT-ROD$(p, 10)$ would return the following arrays:

i	0	1	2	3	4	5	6	7	8	9	10
$r[i]$	0	1	5	8	10	13	17	18	22	25	30
$s[i]$	0	1	2	3	2	2	6	1	2	3	10

MONTCLAIR
STATE UNIVERSITY

CSITBG2 – Data Structures and Algorithms: Java



Longest Common Subsequence (LCS) Problem

Table of Contents

- Dynamic Programming
- Rod-cutting Problem
- **Longest Common Subsequence Problem**

Problem Statement

- Longest Common Subsequence (LCS): Given two strings S1 and S2, find S3 such that
  - S3 appears in both of S1 and S2.
  - It is not necessary that the characters in S3 appear consecutively in S1 and S2.
- Example:
  - Let

$$X = \langle A, B, C, B, D, A, B \rangle$$
$$Y = \langle B, D, C, A, B, A \rangle$$

  - Possible LCS:

$$\langle B, C, B, A \rangle$$

- **Input:** two sequences $X = \langle x_1, x_2, \ldots, x_m \rangle$ and $Y = \langle y_1, y_2, \ldots, y_n \rangle$
- **Output:** a maximum length common subsequence of X and Y.

Optimal Substructure

- Let $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$ be sequences, and let $Z = \langle z_1, z_2, \dots, z_k \rangle$ be any LCS of X and Y.
  - If $x_m = y_n$, then $z_k = x_m = y_n$ and Z_{k-1} is an LCS of X_{m-1} and Y_{n-1}.
  - If $x_m \neq y_n$, then $z_k \neq x_m$ implies that Z is an LCS of X_{m-1} and Y.
  - If $x_m \neq y_n$, then $z_k \neq y_n$ implies that Z is an LCS of X and Y_{n-1}.
- The optimal substructure property:
  - Let $c[i, j]$ be the length of an LCS of the sequences $X[1, \dots, i]$ and $Y[1, \dots, j]$.

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0\,, \\ c[i-1, j-1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j\,, \\ \max(c[i, j-1], c[i-1, j]) & \text{if } i, j > 0 \text{ and } x_i \neq y_j\,. \end{cases}$$

Bottom-up DP Solution (Tabulation)

```
LCS-LENGTH(X, Y)
m = X.length
n = Y.length
let b[1..m, 1..n] and c[0..m, 0..n] be new tables
for i = 1 to m
    c[i, 0] = 0
for j = 0 to n
    c[0, j] = 0
for i = 1 to m
    for j = 1 to n
        if x_i == y_j
            c[i, j] = c[i − 1, j − 1] + 1
            b[i, j] = "↖"
        elseif c[i − 1, j] ≥ c[i, j − 1]
            c[i, j] = c[i − 1, j]
            b[i, j] = "↑"
        else c[i, j] = c[i, j − 1]
            b[i, j] = "←"
return c and b
```

- Example: $X = \langle A, B, C, B, D, A, B \rangle$ and $Y = \langle B, D, C, A, B, A \rangle$

i	j	0	1	2	3	4	5	6
		y_j	B	D	C	A	B	A
0	x_i	0	0	0	0	0	0	0
1	A	0	↑0	↑0	↑0	↖1	←1	↖1
2	B	0	↖1	←1	←1	↑1	↖2	←2
3	C	0	↑1	↑1	↖2	←2	↑2	↑2
4	B	0	↖1	↑1	↑2	↑2	↖3	←3
5	D	0	↑1	↖2	↑2	↑2	↑3	↑3
6	A	0	↑1	↑2	↑2	↖3	↑3	↖4
7	B	0	↖1	↑2	↑2	↑3	↖4	↑4

Solution Reconstruction

```
PRINT-LCS(b, X, i, j)
if i == 0 or j == 0
    return
if b[i, j] == "↖"
    PRINT-LCS(b, X, i − 1, j − 1)
    print x_i
elseif b[i, j] == "↑"
    PRINT-LCS(b, X, i − 1, j)
else PRINT-LCS(b, X, i, j − 1)
```

Reading

- Chapter 15.1, 15.4

MONTCLAIR
STATE UNIVERSITY